REINFORCE Zero: Kernel Alignment Unlocks Stable Asynchronous RL with Importance Sampling

Jian Hu[1], **Hao Zhang**[1], **Yifan Zhang**[1], **Binfeng Xu**[1], **Ao Shen**[2], **Shaokun Zhang**[1], **Yunheng Zou**[1], **Yan Bai**[1], **Jan Kautz**[1] **and Yi Dong**[1]
[1]NVIDIA, [2]Inferact

Abstract

Asynchronous reinforcement learning (RL) scales language-model post-training by overlapping generation and optimization. The rollout engine and the learner then disagree on token probabilities for two reasons: rollouts use older weights (*weight lag*), and the two systems run different GPU kernels, the low-level routines behind matrix multiplication, attention, and normalization, whose floating-point results differ even at identical weights (*kernel mismatch*). We introduce **REINFORCE Zero**, a minimal asynchronous RL recipe. Trainer–inference invariance (TII) aligns the kernels so that both systems return the same token probabilities; *Zero* refers to this zero kernel mismatch. Unclipped token-level importance sampling (IS), the PPO probability ratio without clipping, then corrects weight lag, without sample-rejection gates. With aligned kernels, unclipped IS keeps the remaining weight-lag mismatch smooth and controlled; with standard, unaligned kernels, importance weights fluctuate widely and the rollout–learner discrepancy grows despite IS. We derive the update variance and stability boundary in a Gaussian model, showing how mismatch can destabilize learning even when IS keeps the mean gradient correct. A reusable skill for AI coding agents automates kernel porting, correctness validation, and performance optimization across architectures. On DeepSeek-R1-Distill-Qwen-1.5B, REINFORCE Zero improves reasoning accuracy through **20K policy updates**, achieving quality comparable to the synchronous TII baseline (Sync TII). Asynchronous execution delivers **2.40**× its training-step throughput. Increasing policy lag from 2 to 8 updates preserves stable 20K-update training and final reasoning accuracy. With Qwen3-30B-A3B and Python tools, long-horizon training improves four-benchmark macro pass@1 from 31.29% to 71.55% over 3,000 updates at a fixed 16K evaluation context. On OSWorld, REINFORCE Zero improves Nemotron-3-Nano-Omni-30B's evaluation accuracy from 50.97% to 56.16% after about 400 updates. For this 49K-token workload with complex image inputs, mean end-to-end training-step time is only 6.8% higher than the standard-kernel FlashREINFORCE baseline.

1. Introduction

Asynchronous reinforcement learning scales language-model post-training by overlapping generation with policy optimization (Fu et al., 2025; Hu et al., 2026a). This overlap is especially useful for reasoning and tool use, where variable response lengths and interaction times otherwise leave hardware idle. Sustaining these efficiency gains requires reliable learning from stale trajectories over long training horizons while preserving the quality of synchronous optimization.

The rollout engine and the learner can assign different probabilities to the same token. PPO and GRPO limit the effect of this mismatch by clipping the probability ratio (Schulman et al., 2017; Shao et al., 2024); recent approaches add importance-weight truncation, sample masks, trust regions, or score correction (Hu et al., 2026a; Li et al., 2026; Liu et al., 2025; Marek & Ryabinin, 2026; Yao et al., 2025). In parallel, batch-invariant (BI) kernels, which make a sequence's output independent of the other sequences in its batch, improve numerical consistency across training and inference (He & Thinking Machines Lab, 2025).

Contact: jianh@nvidia.com, yidong@nvidia.com
© 2026 NVIDIA. All rights reserved.

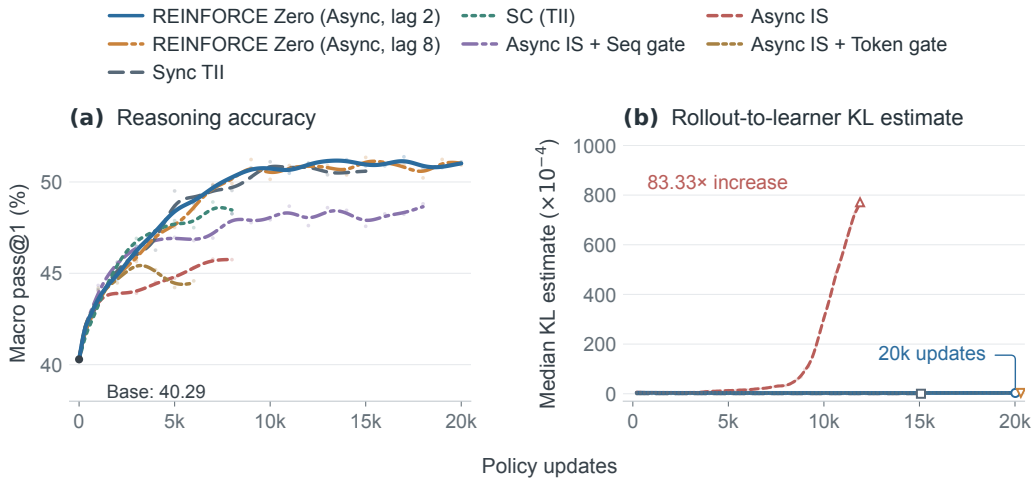


Figure 1: Reasoning quality and rollout–learner discrepancy. (a) Five-benchmark macro pass@1 with equal benchmark weights and an 8K response budget; all methods start at 40.29%. (b) Median sampled rollout-to-learner KL in 200-update windows, evaluated before each learner update (Appendix A.3). Points show measurements; curves are smoothed visual guides. REINFORCE Zero uses TII and token-level IS; SC (TII) uses Score Centering without IS. Sync TII is the only synchronous method. The three IS baselines use standard kernels. The annotation in (b) compares the Async IS medians over updates 1–1,000 and 10,001–11,000; Figure 5 shows its reward and entropy over the same period.

We distinguish two sources of rollout–learner mismatch. *Weight lag* arises because asynchronous rollouts use older weights than the learner. *Kernel mismatch* arises because the two systems compute the same network with different kernels: the low-level GPU routines for operations such as matrix multiplication, attention, and normalization. Floating-point results depend on the kernel and on how sequences are batched, so kernel mismatch can persist even at identical weights (Yao et al., 2025) and destabilize nominally on-policy RL (Liu et al., 2025; Zhong et al., 2026). Limiting rollout delay alone therefore does not remove this source of instability. We call kernels *standard* when each system runs its default implementation and *aligned* when both return matching token probabilities at identical weights and histories.

Our importance-sampling (IS) diagnostics reveal a central distinction: unclipped token-level IS controls the smoother mismatch from weight lag, whereas kernel mismatch produces random numerical fluctuations that persist despite IS. With aligned kernels, unclipped IS keeps the sampled rollout-to-learner KL estimate flat and learning stable (Figure 1b). Section 3 examines this contrast and its implications for asynchronous RL.

Building on this finding, we introduce **REINFORCE Zero**, a minimal asynchronous RL recipe that combines trainer–inference invariance (TII) with unclipped token-level IS, without sample-rejection gates, which discard sequences or tokens whose rollout–learner divergence exceeds a threshold. TII ensures matching token probabilities between the learner (trainer) and the rollout (inference) engine at identical weights and histories; *Zero* refers to this zero kernel mismatch. IS corrects the remaining weight lag. The update is the PPO/GRPO surrogate with a group-mean baseline and an unclipped ratio. An AI coding agent, guided by a reusable skill, automates kernel alignment and performance optimization to implement the recipe efficiently.

Prior work aligned kernels to recover on-policy RL (He & Thinking Machines Lab, 2025) and paired asynchronous alignment with clipped updates (Wang & TorchTitan Team, 2026). We show that with

aligned kernels, unclipped IS sustains asynchronous learning without clipping or rejection, through 20K updates and at lags up to 64.

A Gaussian analysis shows how mismatch amplifies update variance and narrows the stable step-size range even under unbiased IS correction.

Our contributions are fourfold:

1. **A minimal recipe for stable asynchronous RL.** REINFORCE Zero combines TII with unclipped token-level IS. With kernel mismatch controlled, unclipped IS corrects weight lag and sustains stable asynchronous learning without rejection gates.
2. **How mismatch amplifies optimization error.** We separate weight lag from kernel mismatch and derive an exact error recurrence and stability condition in a Gaussian model. Even fixed-magnitude mismatch can turn bounded squared error into growing error under unbiased IS correction.
3. **Stable asynchronous learning over 20K policy updates.** We demonstrate stable learning at policy lags 2 and 8, reasoning quality comparable to Sync TII, and $2.40\times$ its training-step throughput. The recipe also sustains learning at lag 64 with Qwen3-4B, with a 30B MoE model and Python tools, and in visual-agent training on OSWorld.
4. **Automated kernel alignment.** We provide a reusable skill for AI coding agents[1] for kernel porting, correctness validation, and performance optimization across attention, MoE, and Mamba hybrid architectures (Appendix A.1).

2. Problem Setting

We consider asynchronous reinforcement learning for language-model agents. A task $x \sim \mathcal{D}$ initializes a reasoning or tool-interaction episode. At each policy-token step, $\pi_\theta(a_t \mid h_t)$ generates an action token from the history h_t, which contains the prompt, previous generated tokens, and any tool or environment observations. A completed trajectory τ receives a scalar terminal reward $R(x, \tau)$. The objective is $J(\theta) = \mathbb{E}_{x\sim\mathcal{D},\, \tau\sim\pi_\theta(\cdot|x)}[R(x, \tau)]$, with environment dynamics implicit in the trajectory law. Only policy-generated action tokens contribute to the update.

Rollout workers use periodically refreshed policy snapshots while the learner continues updating. A batch contains B completed trajectories and may span multiple behavior versions. Each action token retains its actual sampling probability $\mu_v(a_t \mid h_t)$, where $v \le k$ is the behavior version and k is the current learner update. These stored probabilities define the denominator of the importance ratio. Policy lag $k - v$ counts the learner updates between the weights that generated a token and the weights being updated; synchronous training has lag zero. Experiments are labeled by their nominal pipeline lag (2, 8, or 64).

Three policies separate two sources of discrepancy. For each token, fix its history. Three next-token distributions are involved:

Policy	Weights	Computed by
π_{θ_k}	current, θ_k	learner
π_{θ_v}	rollout, θ_v	learner
μ_v	rollout, θ_v	rollout engine, recorded at sampling

[1] https://github.com/NVIDIA-NeMo/labs-molt/tree/main/.agents/skills/train-inference-consistency

Both learner evaluations share one execution context (kernels and batch composition); μ_v is the actual rollout law in the rollout engine's context. Thus π_{θ_k} versus π_{θ_v} changes only the weights and isolates weight lag, while π_{θ_v} versus μ_v changes only the execution and isolates kernel mismatch. We suppress the history in the notation and assume common positive support and matching sampling transformations (such as temperature and top-p) for the exact identities below. The log ratio is

$$\log \frac{\pi_{\theta_k}}{\mu_v} = \underbrace{\log \frac{\pi_{\theta_k}}{\pi_{\theta_v}}}_{\text{weight lag}} + \underbrace{\log \frac{\pi_{\theta_v}}{\mu_v}}_{\text{kernel mismatch}} . \tag{1}$$

At lag $k - v \leq L$, the first term sums the changes from at most L recent updates. The second compares the rollout and learner implementations and can change with the weights, histories, or batching context. Synchronization removes the first term; TII removes the second through batch invariance (a sequence's output does not depend on the rest of its batch) and matched forward computation (both systems evaluate the same operators with the same kernels).

3. Kernel alignment unlocks unclipped importance sampling

Weight lag and kernel mismatch behave differently over training. When policy updates are small, weight lag follows a smoothly changing policy trajectory. Kernel mismatch depends on batch composition and numerical execution, introducing fluctuations even when rollout and learner use identical weights (Figure 2a).

In the DeepSeek-R1-Distill-Qwen-1.5B reasoning experiments, aligned kernels allow unclipped token-level IS to keep the remaining weight-lag mismatch smooth and controlled. With standard kernels, large importance-weight fluctuations persist despite IS. The importance-weight diagnostic D_3 averages $w - 1 - \log w$ over the action tokens of an update; it is zero when every importance weight w equals one and grows as the weights spread (Appendix A.3). Its peak-to-median ratio over training is 442.54 under standard-kernel IS and 1.50 under aligned kernels (Figure 2b; Table 2). The sampled rollout-to-learner KL stays flat at policy lags 2 and 8 with aligned kernels, while standard-kernel IS exhibits growing discrepancy (Figure 1b).

These observations lead to a simple recipe: remove kernel mismatch, then correct the remaining weight lag. Trainer–inference invariance (TII) matches token probabilities at identical weights and histories, eliminating the kernel component of Equation (1). REINFORCE Zero combines TII with unclipped token-level IS to sustain asynchronous learning without sample-rejection gates.

4. REINFORCE Zero

4.1. Importance-weighted policy update

For each prompt, collect G responses with terminal rewards R_i and set $A_i = R_i - G^{-1} \sum_{j \in \mathcal{G}(i)} R_j$, where $\mathcal{G}(i)$ contains the responses to the same prompt as response i. For a batch of B responses, let T_i be the number of valid (policy-generated) action tokens in response i, with histories h_{it}, actions a_{it}, and recorded behavior probabilities $\mu_i(a_{it} \mid h_{it})$, the rollout distribution μ_v of the version that generated response i. We write the importance-weighted loss in sample-mean form:

$$\widehat{\mathcal{L}}(\theta) = -\frac{1}{B} \sum_{i=1}^{B} \frac{1}{T_i} \sum_{t=1}^{T_i} A_i \, \frac{\pi_\theta(a_{it} \mid h_{it})}{\mu_i(a_{it} \mid h_{it})}. \tag{2}$$

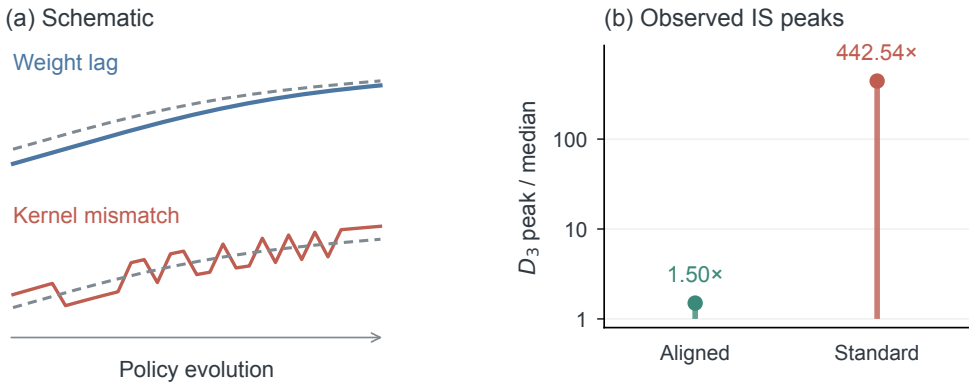


Figure 2: Kernel alignment suppresses importance-weight fluctuations. (a) Schematic: the rollout policy (solid) against the learner reference (dashed) under smoother weight lag and under fluctuating, execution-dependent kernel mismatch. (b) Observed peak-to-median ratio of the batch-averaged importance-weight diagnostic D_3 for aligned REINFORCE Zero and standard-kernel IS (Table 2), on a logarithmic scale.

The loss averages over valid action tokens within each response, then over responses. It is the PPO/GRPO surrogate (Schulman et al., 2017; Shao et al., 2024) with the ratio left unclipped: the importance ratio multiplies a group-mean advantage without standard-deviation normalization. This importance-weighted policy-gradient update follows the REINFORCE family of estimators (Ahmadian et al., 2024; Shao et al., 2024; Williams, 1992). We optimize this loss by differentiating the numerator while holding advantages, recorded behavior probabilities, and response lengths fixed.

Token-level IS reweights conditional actions under the fixed rollout-history distribution (Schulman et al., 2015; Shao et al., 2024).

4.2. Why use the full IS correction

At a fixed history h, let $p = \pi_\theta(\cdot \mid h)$ be the target policy and $q = \mu(\cdot \mid h)$ the actual sampling policy on common positive support. Write $s(a) = \nabla_\theta \log p(a)$ and hold the advantage $A(a)$ fixed when differentiating. Expectations below condition on h; detached response/token weights can be absorbed into A.

The target score has zero mean, $\mathbb{E}_p[s] = 0$, so the target update is the covariance between the advantage and the score. Under the sampling policy, the uncorrected update also contains score-mean drift:

$$\begin{aligned} g^\star &= \mathbb{E}_p[As] = \mathrm{Cov}_p(A, s), \\ \mathbb{E}_q[As] &= \mathrm{Cov}_q(A, s) + \underbrace{\mathbb{E}_q[A]\mathbb{E}_q[s]}_{\text{score-mean drift}} . \end{aligned} \tag{3}$$

Score Centering (SC) (Marek & Ryabinin, 2026) subtracts the sampler-expected score, removing this drift but retaining the covariance under q:

$$\begin{aligned} g_{\mathrm{SC}} &= \mathbb{E}_q[A(s - \mathbb{E}_q[s])] = \mathrm{Cov}_q(A, s), \\ g_{\mathrm{SC}} - g^\star &= \mathrm{Cov}_q(A, s) - \mathrm{Cov}_p(A, s). \end{aligned} \tag{4}$$

Unclipped IS, with $w = p/q$, restores both the zero-mean target score (Marek & Ryabinin, 2026, Appendix A.2) and the target covariance:

$$\begin{aligned} \mathbb{E}_q[ws] &= \mathbb{E}_p[s] = \sum_a \nabla_\theta p(a) = 0, \\ g_{\mathrm{IS}} &= \mathbb{E}_q[wAs] = \mathbb{E}_p[As] = \mathrm{Cov}_p(A, s) = g^\star. \end{aligned} \tag{5}$$

It therefore removes both score-mean drift and covariance bias. Clipping the weights generally introduces bias because $\mathbb{E}_q[w_{\mathrm{clip}} As] \neq \mathrm{Cov}_p(A, s)$ for the clipped weights w_{clip}.

TII makes this full correction practical by suppressing the correction spikes observed with standard kernels (Section 3). Unclipped IS then corrects the remaining weight lag. Section 5 explains why these spikes matter even when IS preserves the mean gradient.

4.3. Establishing trainer–inference invariance

TII has two parts. *Batch invariance*: the rollout engine returns bitwise-identical log-probabilities for a sequence across batch composition, prefill and decode, cache states, and graph or eager execution. *Matched forward computation*: the learner evaluates the same operators with the same kernels (matrix multiplication, attention, RMS normalization, SwiGLU, rotary position encoding, and MoE layers with routing replay), so that replaying the same weights and tokens reproduces the saved rollout log-probabilities. Custom autograd wrappers supply the backward pass.

An AI coding agent establishes TII by following a reusable skill in three stages.

1. Prepare the reference. Map the operations, establish a performance baseline, and build a small proxy that preserves the target model's computation.

2. Align and validate. Match rollout and learner forward passes on the proxy; verify gradients and optimizer updates against an independent reference.

3. Scale and optimize. Scale to the full model, optimize end-to-end performance, and repeat consistency and gradient checks after optimization.

Appendix A.1 details the skill, validation criteria, and kernel implementations.

5. How mismatch amplifies optimization error

A Gaussian model shows how mismatch amplifies update noise and narrows the stable step-size range even when IS preserves the mean gradient. Consider $p_\theta = \mathcal{N}(\theta, \sigma^2)$ with fixed $\sigma > 0$ and reward $R(a) = -(a - a_\star)^2/2$. Write $x_k = \theta_k - a_\star$ and $V_k = \mathbb{E}[x_k^2]$; the expected reward loss relative to the optimum is $V_k/2$.

The sampling policy μ_k has the same variance and a symmetric mean displacement of fixed magnitude $\kappa\sigma$ (sign drawn independently for each batch). With a fixed step size $\eta > 0$, we update $x_{k+1} = x_k + \eta \bar{g}_k$, where $\bar{g}_k$ averages M unclipped-IS REINFORCE estimates using the realized sampling density and a detached reward baseline $J(\theta_k)$. Appendix B.1 specifies the sampling model and gives the full derivation; $\mathbb{E}_k$ and Var_k below condition on the past training history.

1. IS keeps the mean update correct. The importance weight cancels the sampling density, giving

$$\mathbb{E}_k[\bar{g}_k] = -x_k, \qquad \mathbb{E}_k[x_{k+1}] = (1 - \eta)x_k. \tag{6}$$

The mean error tends to zero for $0 < \eta < 2$. This alone does not ensure high reward: positive and negative errors can cancel in the mean, while both contribute to squared error V_k.

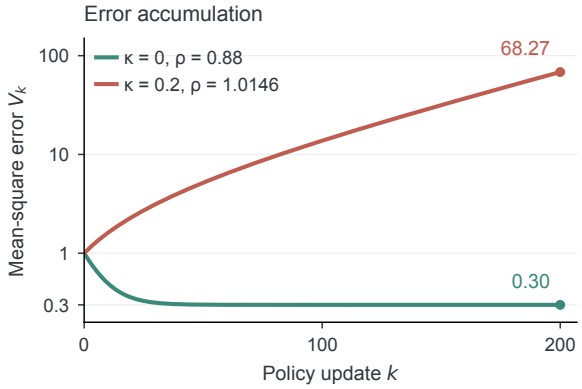


Figure 3: Mismatch can destabilize an unbiased update. Exact expected squared error in the Gaussian model for matched sampling and symmetric mismatch $\kappa = 0.2$, with $\eta = 0.6$, batch size $M = 1$, policy standard deviation $\sigma = 0.2$, and initial error $x_0 = 1$. The vertical axis is logarithmic.

2. Mismatch amplifies update noise. Actions that are rare under the sampling policy but important under the target policy receive large importance weights when sampled. With $w = p_{\theta_k}/\mu_k$, $\mathbb{E}_{\mu_k}[w^2] = e^{\kappa^2}$, and the resulting batch variance is

$$\mathrm{Var}_k(\bar{g}_k) = v_\kappa x_k^2 + b_\kappa. \tag{7}$$

Both coefficients increase with κ and scale as $1/M$ (explicit forms in Appendix B.1). The first term makes updates noisier farther from the optimum; $b_\kappa > 0$ is sampling noise that remains at the optimum.

3. Accumulated noise can increase reward loss. Conditioned on the past, expected squared error equals the squared mean plus the variance. Averaging over the past gives

$$V_{k+1} = \rho V_k + c, \qquad \rho = \underbrace{(1-\eta)^2}_{\text{mean correction}} + \underbrace{\eta^2 v_\kappa}_{\text{error-dependent noise}}, \quad c = \eta^2 b_\kappa. \tag{8}$$

Here ρ multiplies the existing error, while c adds fresh sampling noise at each step. When $\rho < 1$, squared error approaches the finite noise floor $c/(1-\rho)$; at $\rho = 1$ it grows linearly, and at $\rho > 1$ exponentially. The stability condition is

$$0 < \eta < \frac{2}{1 + v_\kappa}.$$

Even at $\kappa = 0$, the upper bound is $2M/(M+2) < 2$ because of REINFORCE sampling noise. Greater mismatch raises v_κ and narrows this range further. Figure 3 shows the same step size producing stable learning at $\kappa = 0$ and growing reward loss at $\kappa = 0.2$, even as both mean parameters approach the optimum. Appendix B.2 gives the numerical calculation.

The model links mismatch magnitude to update variance and the stable step-size range. The experiments identify kernel mismatch as a source of large excursions: the D_3 peak-to-median ratio is 442.54 with standard kernels and 1.50 with aligned kernels (Figure 2b). REINFORCE Zero therefore combines two complementary mechanisms: TII removes the execution-dependent component of Equation (1), while unclipped IS corrects the remaining weight lag.

6. Large-scale RL experiments

REINFORCE Zero sustains stable asynchronous learning across mathematical reasoning, Python tool calls, and visual-agent tasks on OSWorld. Aligned kernels keep rollout-to-learner KL controlled across these settings.

Setup. We implement the asynchronous rollout–learner setup of FlashREINFORCE (Hu et al., 2026a) in the Molt training framework (Hu et al., 2026b). A coding agent (Claude Code with our skill) automates the alignment and performance optimization of batch-invariant kernels across the vLLM rollout engine (vLLM Team, 2026) and the learner. Table 7 in the appendix lists the recipes. *Sync TII* waits for all rollouts before each update, so it has no weight lag and uses no IS; it is the synchronous reference. All other methods use asynchronous execution. The three standard-kernel baselines use token-level IS: *Async IS* applies no rejection; *Async IS + Seq gate* (FlashREINFORCE) discards a response when the mean Bernoulli KL between the rollout and learner probabilities of its sampled tokens exceeds $\gamma = 10^{-2}$ (Appendix A.3); and *Async IS + Token gate* (DPPO) applies this threshold to individual tokens. *SC (TII)* replaces IS with Score Centering (Section 4.2). The main experiments use DeepSeek-R1-Distill-Qwen-1.5B (DeepSeek-AI et al., 2025) and the 1,460-problem MATH training set used by DPPO (Qi et al., 2026). Pass@1 is the mean per-response accuracy over the sampled responses to each problem; macro pass@1 weights benchmarks equally. We evaluate five mathematical reasoning benchmarks and extend the recipe to Qwen3-30B-A3B with Python tools and Nemotron-3-Nano-Omni-30B on OSWorld. Hyperparameters and evaluation protocols appear in Appendix A.2; kernel implementation details appear in Appendix A.1.

6.1. Reasoning quality

REINFORCE Zero achieves reasoning quality comparable to the synchronous reference, Sync TII, at $2.40\times$ its training-step throughput (Section 6.6), and outperforms the asynchronous correction baselines at 6,000 updates (Table 1). Under an 8K response budget, five-benchmark macro pass@1 reaches 48.80% at lag 2 and 48.53% at lag 8, compared with 48.91% for Sync TII, 47.50% for SC (TII), 46.79% for Async IS + Seq gate, 45.32% for Async IS, and 44.59% for Async IS + Token gate. The base model scores 40.29% under the same budget. Figure 1a shows the learning trajectories.

Robustness to increased policy lag. Increasing policy lag fourfold, from 2 to 8, preserves stable training through 20,000 updates. Lag 8 is the nominal pipeline setting; the measured rollout-to-learner lag exceeds it, averaging about 10.5 learner updates with long-response tails reaching 36–38. From 6,000 to 20,000 updates, five-benchmark macro pass@1 improves from 48.80% to 51.10% at lag 2 and from 48.53% to 50.93% at lag 8 (Table 1). The final scores differ by 0.17 percentage points.

6.2. Stable asynchronous learning over 20,000 updates

REINFORCE Zero remains stable through 20,000 asynchronous policy updates using unclipped token-level IS without extra rejection. We compare it with **Sync TII**, which synchronizes each update, and **Async IS**, which uses asynchronous generation and standard kernels.

REINFORCE Zero's sampled rollout-to-learner KL remains flat at both lags 2 and 8, with medians of 3.18×10^{-4} and 3.43×10^{-4}, respectively (Figure 1b). Sync TII stays near zero. Async IS shows a pronounced late-training rise: its median increases **83.33-fold** between its first and last complete 1,000-update windows.

Table 1: Five-benchmark reasoning quality. Pass@1 (%) with an 8K response budget and up to 16 responses per problem. TII and Async indicate whether trainer–inference invariance and asynchronous execution are enabled. All asynchronous runs use lag 2 unless labeled lag 8; lag 8 is nominal, with measured lag averaging about 10.5 updates. The macro mean weights the five benchmarks equally. Sync TII is the synchronous reference; it takes $2.40\times$ the training-step time of REINFORCE Zero (Table 5). The best asynchronous result in each column at 6,000 updates is in bold.

Recipe	TII	Async	MATH 500	AMC23	Olympiad Bench	AIME25	Minerva Math	Macro mean
Before RL (0 updates)								
R1-Distill-1.5B	—	—	68.7	58.1	38.6	21.0	15.0	40.29
6,000 policy updates: synchronous reference								
Sync TII	✓	×	76.75	69.81	49.09	28.75	20.15	48.91
6,000 policy updates: asynchronous								
Async IS	×	✓	71.84	66.88	45.55	27.08	15.23	45.32
Async IS + Seq gate	×	✓	73.88	69.30	47.17	27.43	16.19	46.79
Async IS + Token gate	×	✓	72.10	64.82	45.17	25.97	14.87	44.59
SC (TII)	✓	✓	75.56	70.63	47.18	26.88	17.26	47.50
REINFORCE Zero (lag 2)	✓	✓	76.84	**71.72**	48.72	**29.17**	17.58	**48.80**
REINFORCE Zero (lag 8)	✓	✓	**77.71**	69.53	**49.07**	28.13	**18.20**	48.53
20,000 policy updates: asynchronous								
REINFORCE Zero (lag 2)	✓	✓	78.69	74.69	50.25	31.25	20.63	51.10
REINFORCE Zero (lag 8)	✓	✓	79.43	75.16	50.98	30.00	19.07	50.93

This rise accompanies a late-training decline of Async IS: pass@1 on the AIME24/AIME25 evaluations run during training falls from 34.27%/27.71% to 13.96%/13.02%, token entropy rises, and training reward drops (Figure 5).

REINFORCE Zero's maximum gradient norm before optimizer clipping is 0.0555, compared with 38.27 for Async IS (Figure 4a). The importance-ratio diagnostic D_3 has a maximum close to its median for REINFORCE Zero, while the standard-kernel run exhibits large excursions (Table 2). With TII removing kernel mismatch, the remaining weight-lag mismatch stays smooth and controlled under unclipped IS. Training remains stable for 20K updates at lags 2 and 8, with flat rollout-to-learner KL and controlled gradient norms.

A sequence gate would not change REINFORCE Zero: evaluated as a diagnostic at the same threshold $\gamma = 10^{-2}$, it rejects no sequences in REINFORCE Zero or Sync TII (Figure 4b). For Async IS + Seq gate, mean rejection rises from 0.05% to 24.48% between the first and final 1,000-update windows (Table 2).

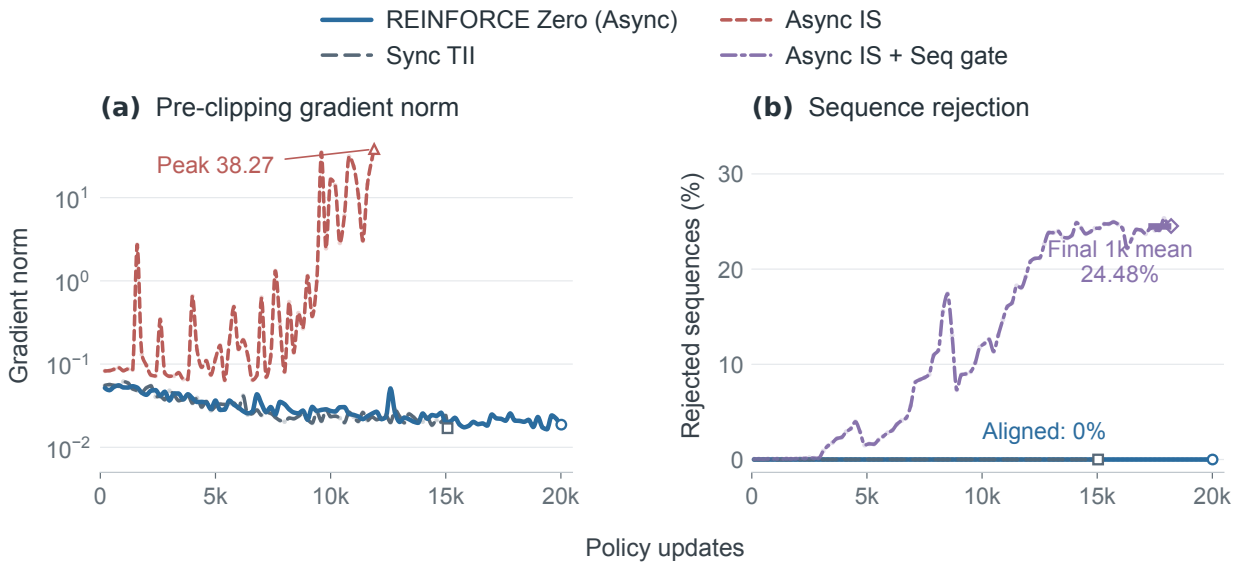


Figure 4: Controlled gradients without sequence rejection. (a) Maximum pre-clipping gradient norm in 200-update windows, on a log scale. (b) Mean sequence rejection in the same windows at threshold $\gamma = 10^{-2}$. Points show window summaries; curves interpolate them. The gate is diagnostic only for REINFORCE Zero and Sync TII and rejects no sequences. Standard-kernel Async IS in (a) is ungated; Async IS + Seq gate in (b) actively applies the gate.

Table 2: Controlled corrections without discarding sequences. D_3 is the batch-averaged importance-ratio diagnostic defined in Appendix A.3; peak / median and gradient maxima use all recorded updates. REINFORCE Zero (lag 2) applies the gate as a diagnostic; Async IS is ungated. Dashes denote unreported or inapplicable statistics.

Statistic	REINFORCE Zero	Async IS	Async IS + Seq gate
Sequence rejection, first 1,000 updates	0%	—	0.05%
Sequence rejection, final 1,000 updates	0%	—	24.48%
D_3 peak / median	1.50×	442.54×	—
Maximum pre-clipping gradient norm	0.0555	38.27	—

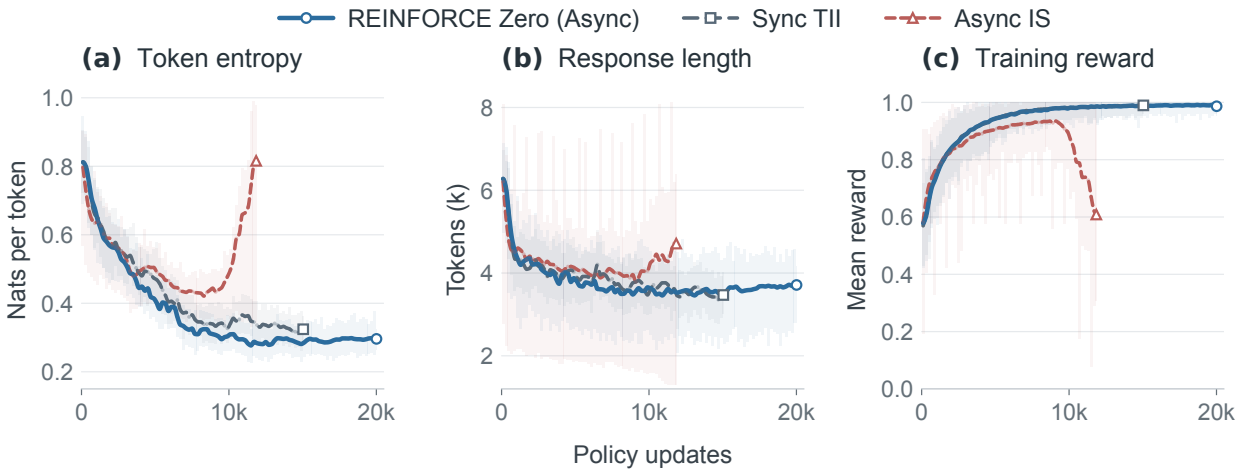


Figure 5: Training dynamics over 20,000 updates. Curves show 200-update means; shading spans each window's minimum and maximum. Token entropy uses the learner's full vocabulary on response positions.

6.3. Stable learning at lag 64

REINFORCE Zero sustains learning at policy lag 64 for 6,000 updates with Qwen3-4B in non-thinking mode and Python tools. Mean reward rises from 0.287 in the first 100 updates to 0.782 in the final 100. Figure 6 shows training reward and AIME24/25/26 evaluation through 5,000 updates.

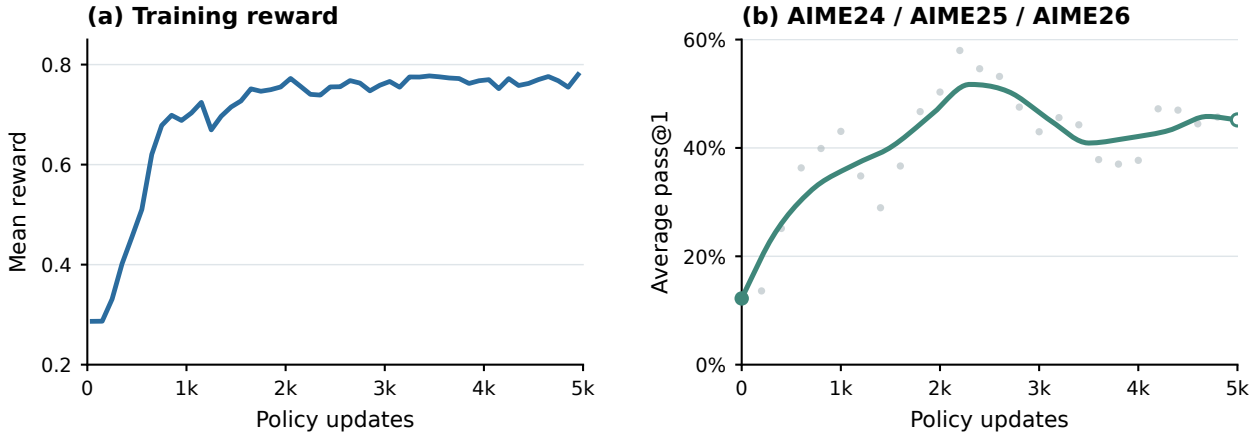


Figure 6: Learning at lag 64 with Qwen3-4B and Python tools. A 16K-context non-thinking run through 5,000 updates: (a) training reward as 100-update means; (b) equal-weight AIME24/AIME25/AIME26 pass@1 at 200-update checkpoints (points) and a smoothed trend from 400-update bins (curve).

6.4. Long-horizon learning with Qwen3-30B-A3B and tools

With Qwen3-30B-A3B and Python tools, REINFORCE Zero improves four-benchmark macro pass@1 from 31.29% before RL to 65.78% after 500 updates and 71.55% after 3,000 updates at a fixed 16K evaluation context (Table 3). With a 32K evaluation context window, REINFORCE Zero achieves 72.64% macro pass@1 across four benchmarks and 77.66% mean pass@1 on AIME25/AIME26 (Table 3). Mean training reward rises from 0.566 to 0.904, and training remains stable beyond 3.5K updates. The sampled rollout-to-learner KL estimate levels off near 1.1×10^{-3} and remains below 1.31×10^{-3} across all updates (Figure 7c).

We train the non-thinking mixture-of-experts (MoE) model on 7,500 problems. The recipe combines TII with group-mean advantages and unclipped token-level IS, with no sequence gate. Batch-invariant kernels and Rollout Routing Replay (R3) (Ma et al., 2025) align numerical execution and expert selection.

Table 3: Qwen3-30B-A3B with Python tools: benchmark results. Pass@1 (%); the macro mean gives equal weight to AIME25, AIME26, AMC23, and Minerva Math.

	16K context			32K context
Benchmark	Before RL	Update 500	Update 3,000	Final
AIME25	21.25	63.75	74.79	76.25
AIME26	25.00	67.92	75.62	79.06
AMC23	60.31	94.69	99.14	98.75
Minerva Math	18.61	36.76	36.63	36.51
Macro mean	31.29	65.78	71.55	72.64

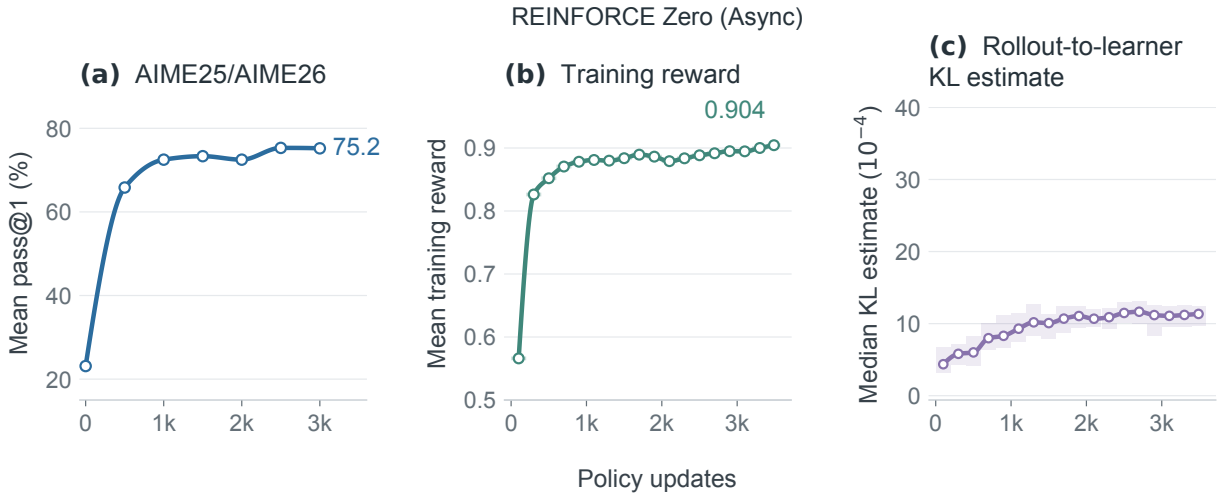


Figure 7: Learning and stability with Qwen3-30B-A3B and Python tools. Asynchronous REINFORCE Zero training: (a) mean AIME25/AIME26 pass@1 with a 16K context window, (b) training reward and (c) the sampled rollout-to-learner KL estimate. Panel (b) shows 200-update means; (c) shows 200-update medians. Shading spans each window's observed per-update KL estimates; curves are visual guides.

6.5. Visual-agent training on OSWorld

On OSWorld, REINFORCE Zero improves Nemotron-3-Nano-Omni-30B from 50.97% to 56.16% task-weighted pass@1 after 428 updates, a gain of 5.19 percentage points (Table 4). The sampled rollout-to-learner KL curve stays flat throughout training (Figure 8). This hybrid model combines Mamba-2, attention, and MoE layers (NVIDIA, 2026). The agent completes OSWorld desktop tasks (Xie et al., 2024) by observing screenshots and executing mouse and keyboard actions through PyAutoGUI. Its 49K-token context combines task instructions, multi-turn interaction histories, and up to 20 screenshots.

REINFORCE Zero uses TII and unclipped token-level IS; the FlashREINFORCE baseline is the Async IS + Seq gate recipe: standard kernels and token-level IS with the Bernoulli-KL sequence gate at threshold 10^{-2}. Appendix A.2 gives the training and evaluation settings.

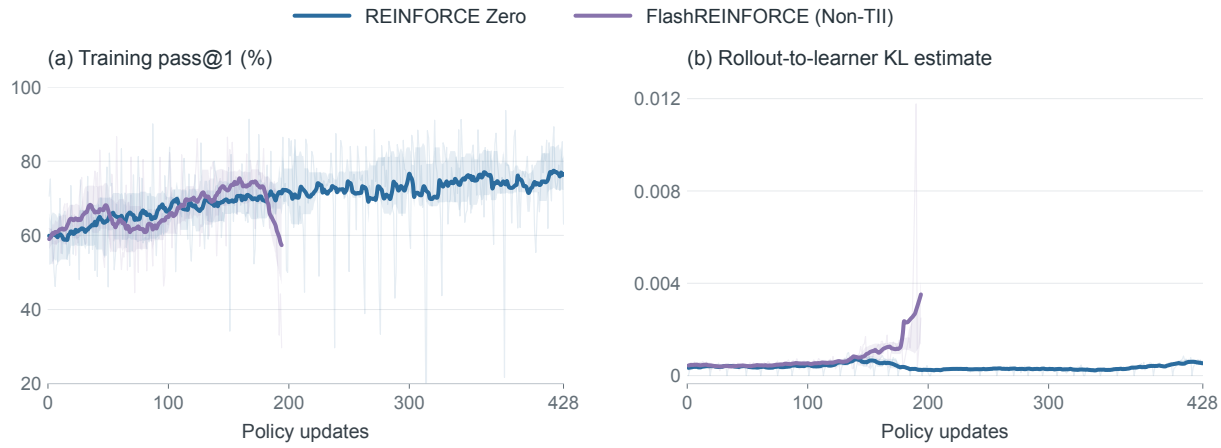


Figure 8: OSWorld training. (a) Training pass@1 and (b) the sampled rollout-to-learner KL estimate for REINFORCE Zero and FlashREINFORCE through updates 428 and 194, respectively. Faint lines show recorded updates; bold lines show 21-update moving averages. Shading shows the interquartile range within each window.

Table 4: OSWorld evaluation with Nemotron-3-Nano-Omni-30B. Task-weighted pass@1.

Method	Policy updates	Accuracy (%)
Base model	0	50.97
REINFORCE Zero	428	**56.16**

6.6. Training efficiency

Asynchronous training. REINFORCE Zero achieves $2.40\times$ the training-step throughput of Sync TII by overlapping generation and optimization (Table 5), with comparable reasoning quality (Section 6.1). Both use TII and 16 GPUs; timing covers 12,000 updates.

Table 5: Training-step cost at equal update count. Total training-step time for 12,000 updates on 16 GPUs, with TII enabled in both runs, excluding evaluation. REINFORCE Zero runs at lag 2.

Recipe	Training hours	Seconds/update	Speedup vs. Sync TII
Sync TII	164.77	49.43	$1.00\times$
REINFORCE Zero	**68.55**	**20.56**	$\mathbf{2.40}\times$

Alignment cost. For Nemotron-3-Nano-Omni-30B, mean end-to-end step time is 944.25 s for REINFORCE Zero and 884.17 s for the FlashREINFORCE (Async IS + Seq gate) baseline (Table 6), a 6.8% increase in step time. In separate architecture benchmarks, the same reusable workflow aligns DeepSeek-V4-Flash and NVFP4 Nemotron-3.5, with rollout throughput reductions of 8.7% and 12.5%, respectively.

Table 6: Cost of aligned execution. Changes are relative to the corresponding unaligned baseline. Appendix A.1 describes the additional model checks.

Model / workload	Metric	Change vs. baseline
Nemotron-3-Nano-Omni-30B / OSWorld	End-to-end step time	6.8% higher
DeepSeek-V4-Flash	Rollout throughput	8.7% lower
Nemotron-3.5 / SWE-bench Pro	Rollout throughput	12.5% lower

7. Related Work

Asynchronous RL systems. IMPALA couples decoupled actors and learners with V-trace off-policy correction (Espeholt et al., 2018). LLM systems use asynchronous rollouts to improve utilization and accommodate variable trajectory durations (Fu et al., 2025; Huang et al., 2026). Asynchronous RLHF separates generation from learning and studies tolerance to stale data (Noukhovitch et al., 2024); PipelineRL refreshes weights during generation to improve data freshness (Piché et al., 2025). We study numerical alignment with unclipped token-level IS over long asynchronous training horizons.

Algorithmic correction of policy mismatch. PPO and GRPO clip the probability ratio (Schulman et al., 2017; Shao et al., 2024). TIS caps large importance weights (Yao et al., 2025); MIS and TRM use sequence-level masking to restrict off-policy updates (Li et al., 2026; Liu et al., 2025). IcePop uses importance-ratio acceptance intervals (Ling Team et al., 2025), and DPPO uses divergence-based, direction-aware masks (Qi et al., 2026). Score Centering subtracts the sampler-expected score to

cancel conditional score drift (Marek & Ryabinin, 2026). Confidence-aware IS studies confidence-dependent correction and second-moment tradeoffs (Yu et al., 2026). FlashREINFORCE combines critic-free learning, token-level IS, and sequence-level trust gates (Hu et al., 2026a). Our recipe combines aligned execution with unclipped IS for sustained asynchronous learning without additional gates.

Numerical consistency across training and inference. Thinking Machines Lab (TML) introduced batch-invariant kernels and demonstrated their role in recovering on-policy RL (He & Thinking Machines Lab, 2025). FP16 training offers another route to reducing mismatch (Qi et al., 2025). R3 aligns expert selection by replaying rollout routing decisions during MoE training (Ma et al., 2025). TorchTitan experiments study asynchronous numerical alignment with clipped updates (Wang & TorchTitan Team, 2026). VeXact uses a zero-mismatch reference to isolate discrepancy-induced optimization effects and evaluate corrections (Zhong et al., 2026). We pair alignment with unclipped token-level IS, supported by an analysis of mismatch-induced error accumulation, 20,000-update experiments, and an automated alignment workflow across model architectures.

8. Conclusion

REINFORCE Zero enables stable asynchronous RL by establishing TII and correcting weight lag with unclipped token-level IS. Controlling kernel mismatch allows unclipped IS to sustain learning without rejection gates. Our Gaussian analysis shows how mismatch amplifies update variance and narrows the stable step-size range despite an unbiased mean gradient.

The recipe delivers reasoning quality comparable to Sync TII at $2.40\times$ its training-step throughput. Reasoning accuracy improves through 20,000 updates at policy lags 2 and 8, with final scores differing by 0.17 percentage points. With Python tools, Qwen3-4B learns at lag 64, and Qwen3-30B-A3B improves macro pass@1 from 31.29% to 71.55% over 3,000 updates. On OSWorld, the recipe raises visual-agent accuracy from 50.97% to 56.16%. A reusable skill for AI coding agents automates kernel alignment across attention, MoE, and Mamba hybrid architectures, making the recipe practical across these workloads.

References

Arash Ahmadian, Chris Cremer, Matthias Gallé, Marzieh Fadaee, Julia Kreutzer, Olivier Pietquin, Ahmet Üstün, and Sara Hooker. Back to basics: Revisiting REINFORCE-style optimization for learning from human feedback in LLMs. In *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 12248–12267. Association for Computational Linguistics, 2024. doi: 10.18653/v1/2024.acl-long.662. URL https://aclanthology.org/2024.acl-long.662/. 5

DeepSeek-AI et al. DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning, 2025. URL https://arxiv.org/abs/2501.12948. 8

Lasse Espeholt, Hubert Soyer, Remi Munos, Karen Simonyan, Vlad Mnih, Tom Ward, Yotam Doron, Vlad Firoiu, Tim Harley, Iain Dunning, Shane Legg, and Koray Kavukcuoglu. IMPALA: Scalable distributed deep-RL with importance weighted actor-learner architectures. In *Proceedings of the 35th International Conference on Machine Learning*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1407–1416. PMLR, 2018. URL https://proceedings.mlr.press/v80/espeholt18a.html. 13

Wei Fu, Jiaxuan Gao, Xujie Shen, Chen Zhu, Zhiyu Mei, Chuyi He, Shusheng Xu, Guo Wei, Jun Mei, Jiashu Wang, Tongkai Yang, Binhang Yuan, and Yi Wu. AReaL: A large-scale asynchronous reinforcement learning system for language reasoning, 2025. URL https://arxiv.org/abs/2505.24298. 1, 13

Horace He and Thinking Machines Lab. Defeating nondeterminism in LLM inference. *Thinking Machines Lab: Connectionism*, 2025. doi: 10.64434/tml.20250910. URL https://thinkingmachines.ai/blog/defeating-nondeterminism-in-llm-inference/. 1, 2, 14

Jian Hu, Yifan Zhang, Hao Zhang, Binfeng Xu, Shaokun Zhang, Hongqing Peng, Zhiding Yu, Pavlo Molchanov, Jan Kautz, and Yi Dong. FlashREINFORCE: Critic-free single-rollout asynchronous RL for agentic language models, 2026a. URL https://yifanzhang-pro.github.io/FlashREINFORCE/FlashREINFORCE.pdf. 1, 8, 14, 18

Jian Hu, Huiying Li, Hao Zhang, Binfeng Xu, Yifan Zhang, Shaokun Zhang, Hemil Desai, Michael Demoret, Pavlo Molchanov, Jan Kautz, and Yi Dong. Molt: A scalable PyTorch-native training framework for agentic reinforcement learning, 2026b. URL https://arxiv.org/abs/2607.21653. 8

Luke J. Huang, Zhuoyang Zhang, Qinghao Hu, Shang Yang, and Song Han. Stable asynchrony: Variance-controlled off-policy RL for LLMs, 2026. URL https://arxiv.org/abs/2602.17616. 13

Yingru Li, Jiacai Liu, Jiawei Xu, Yuxuan Tong, Ziniu Li, Qian Liu, and Baoxiang Wang. Trust region masking for long-horizon LLM reinforcement learning. *arXiv preprint arXiv:2512.23075*, version 5, 2026. URL https://arxiv.org/abs/2512.23075v5. 1, 13

Ling Team et al. Every step evolves: Scaling reinforcement learning for trillion-scale thinking model, 2025. URL https://arxiv.org/abs/2510.18855. 13

Jiacai Liu, Yingru Li, et al. When speed kills stability: Demystifying RL collapse from the training-inference mismatch. Technical report, September 2025. URL https://richardli.xyz/rl-collapse. 1, 2, 13

Wenhan Ma, Hailin Zhang, Liang Zhao, Yifan Song, Yudong Wang, Zhifang Sui, and Fuli Luo. Stabilizing MoE reinforcement learning by aligning training and inference routers, 2025. URL https://arxiv.org/abs/2510.11370. 11, 14

Martin Marek and Max Ryabinin. Score centering stabilizes off-policy reinforcement learning, 2026. URL https://arxiv.org/abs/2609.20807. 1, 5, 6, 14

Michael Noukhovitch, Shengyi Huang, Sophie Xhonneux, Arian Hosseini, Rishabh Agarwal, and Aaron Courville. Asynchronous RLHF: Faster and more efficient off-policy RL for language models, 2024. Published at ICLR 2025. URL https://arxiv.org/abs/2410.18252. 13

NVIDIA. Nemotron 3 Nano Omni: Efficient and open multimodal intelligence, 2026. URL https://arxiv.org/abs/2604.24954. 12

Alexandre Piché, Ehsan Kamalloo, Rafael Pardinas, Xiaoyin Chen, and Dzmitry Bahdanau. PipelineRL: Faster on-policy reinforcement learning for long sequence generation, 2025. URL https://arxiv.org/abs/2509.19128. 13

Penghui Qi, Zichen Liu, Xiangxin Zhou, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Defeating the training-inference mismatch via FP16, 2025. URL https://arxiv.org/abs/2510.26788. 14

Penghui Qi, Xiangxin Zhou, Zichen Liu, Tianyu Pang, Chao Du, Min Lin, and Wee Sun Lee. Rethinking the trust region in LLM reinforcement learning, 2026. URL https://arxiv.org/abs/2602.04879. 8, 13, 18

John Schulman, Sergey Levine, Pieter Abbeel, Michael Jordan, and Philipp Moritz. Trust region policy optimization. In *Proceedings of the 32nd International Conference on Machine Learning*, volume 37 of *Proceedings of Machine Learning Research*, pp. 1889–1897. PMLR, 2015. URL https://proceedings.mlr.press/v37/schulman15.html. 5

John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. Proximal policy optimization algorithms, 2017. URL https://arxiv.org/abs/1707.06347. 1, 5, 13

Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models, 2024. URL https://arxiv.org/abs/2402.03300. 1, 5, 13

vLLM Team. Batch invariance. *vLLM documentation*, accessed October 2026. URL https://docs.vllm.ai/en/latest/features/batch_invariance/. 8

Yichuan Wang and TorchTitan Team. Defending against the training–inference numeric mismatch in RL (especially linear attention)—and whether it helps async RL. Technical blog, August 2026. URL https://yichuan-w.github.io/blog/GDN-train-inference-mismatch-asyncRL/. 2, 14

Ronald J. Williams. Simple statistical gradient-following algorithms for connectionist reinforcement learning. *Machine Learning*, 8:229–256, 1992. doi: 10.1007/BF00992696. 5

Tianbao Xie, Danyang Zhang, Jixuan Chen, Xiaochuan Li, Siheng Zhao, Ruisheng Cao, Toh Jing Hua, Zhoujun Cheng, Dongchan Shin, Fangyu Lei, Yitao Liu, Yiheng Xu, Shuyan Zhou, Silvio Savarese, Caiming Xiong, Victor Zhong, and Tao Yu. OSWorld: Benchmarking multimodal agents for open-ended tasks in real computer environments, 2024. URL https://arxiv.org/abs/2404.07972. 12

Feng Yao, Liyuan Liu, Dinghuai Zhang, Chengyu Dong, Jingbo Shang, and Jianfeng Gao. On the rollout-training mismatch in modern RL systems. OPT 2025: Optimization for Machine Learning Workshop, 2025. URL https://opt-ml.org/papers/2025/paper116.pdf. 1, 2, 13

Tianrun Yu, Kaixiang Zhao, Shangzhe Li, Yuxiao Yang, Porter Jenkins, Weitong Zhang, and Taylor W. Killian. Rethinking training-inference mismatch in LLM reinforcement learning: Where it arises and how to correct it. *arXiv preprint arXiv:2609.32444*, 2026. URL https://arxiv.org/abs/2609.32444. 14

Tianle Zhong, Neiwen Ling, Yifan Pi, Zijun Wei, Tianshu Yu, Geoffrey Fox, Peng Wu, and Xiao Yu. Diagnosing training inference mismatch in LLM reinforcement learning via a zero-mismatch reference. *arXiv preprint arXiv:2605.14220*, 2026. URL https://arxiv.org/abs/2605.14220v2. 2, 14

A. Implementation and Experimental Details

A.1. Implementing trainer–inference invariance

Our automated kernel-porting workflow targets matching rollout and learner token log-probabilities at the same weights and tokens, with correct training gradients and high throughput. We use the agent skill with Claude Code to port vLLM's batch-invariant kernels to AutoModel.

Its eleven checks (C0–C10) cover forward agreement, gradient correctness, and performance in three phases:

Prepare the reference

- **C0** **Map the operations.** List every module that contributes to token log-probabilities, including both learner scoring paths, and identify the implementation changes needed for alignment.
- **C1** **Fix the baseline.** Choose the workload, precision, and parallel layout, and measure the baseline with batch invariance disabled.
- **C2** **Build a small proxy.** Preserve the target's operator types, dimensions, and state paths in a few representative layers.

Align and validate on the proxy

- **C3** **Check rollout invariance.** Require bitwise-identical log-probabilities across batch composition, prefill and decode, cache states, and graph or eager execution.
- **C4** **Remove avoidable overhead.** Profile both engines and reduce unnecessary launches, synchronization, copies, and allocations.
- **C5** **Match learner and rollout.** Replay the same weights and tokens. Compare saved rollout log-probabilities with both old-policy scoring and the gradient-enabled actor forward pass, including repeated and differently packed inputs.
- **C6** **Specify the backward pass.** Define derivatives for each aligned operator and verify finite gradients.
- **C7** **Validate gradients and updates.** Compare gradients and one optimizer step with an independent reference.
- **C8** **Run two RL updates.** Check rollout–learner agreement before and after two synchronous policy updates.

Scale and verify the final system

- **C9** **Scale gradually.** Increase model and workload size while repeating numerical checks.
- **C10** **Optimize and verify.** Minimize alignment overhead through profiling and optimization, repeating numerical checks after each change. Final acceptance requires at most 20% end-to-end training-throughput loss relative to the unaligned baseline under the same workload.

Kernel implementation. Custom autograd wrappers keep the aligned forward operations and supply backward passes through analytic derivatives or differentiable reference recomputation. Attention uses the same fixed absolute key segments during decoding and full-sequence scoring, with FlashAttention-2 recomputation for backward. The port also covers matrix multiplication, RMS normalization, SwiGLU, and rotary positional encoding. MoE wrappers support the learner's expert-weight layout and routing replay (R3).

For low-precision training, quantization-aware training (QAT) can use the same aligned quantized forward operators, scales, and rounding rules in the learner and rollout, with gradient estimators supporting the backward pass.

Additional model results and alignment cost. The same agent workflow achieves bitwise agreement between rollout and learner token log-probabilities on Nemotron-3-Nano-Omni-30B (Omni3), whose hybrid architecture combines Mamba-2, attention, and MoE layers.

On DeepSeek-V4-Flash, rollout and learner log-probabilities agree exactly across 100 validation steps. The aligned implementation reduces rollout throughput by 8.7% relative to the unaligned baseline. Applying the workflow to Nemotron-3.5, with its Mamba hybrid architecture and NVFP4 precision, reduces rollout throughput by 12.5% during SWE-bench Pro agentic RL.

A.2. Training and evaluation settings

Mathematical reasoning without tools: training. We train DeepSeek-R1-Distill-Qwen-1.5B on the `sail/Sanity-Test-R1D-1.5B` training split used by DPPO (Qi et al., 2026), which contains 1,460 MATH problems. Table 7 summarizes invariance and correction settings. Training uses BF16. Sampling uses temperature 1 and top-p 1, with response/context limits of 8,192/9,216 tokens. Each update uses 128 responses and a sample-mean loss. REINFORCE Zero uses 32 prompts with four responses per prompt and group-mean advantages at lags 2 and 8. Sync TII uses synchronous updates without IS. Both REINFORCE Zero lag settings use a 2+14 learner–rollout GPU split. The learning rate uses 3% warm-up to 10^{-6} followed by cosine decay to 10^{-7}.

Table 7: Training configurations for mathematical reasoning without tools. All use 128 responses per update and peak learning rate 10^{-6}. Gate denotes divergence rejection.

Recipe	Execution	TII	IS	Gate threshold
REINFORCE Zero	Async	Yes	Yes	Off
Sync TII	Sync	Yes	No	Off
Async IS	Async	No	Yes	Off
Async IS + Seq gate	Async	No	Yes	10^{-2}
Async IS + Token gate	Async	No	Yes	10^{-2}
SC (TII)	Async	Yes	No	Off

Correction baselines. Async IS + Seq gate uses FlashREINFORCE (Hu et al., 2026a); Async IS + Token gate uses DPPO (Qi et al., 2026). Their rejection rules operate at the sequence and token level, respectively. Both use token-level IS and standard kernels (thresholds in Table 7). Async IS disables these rejection gates. SC (TII) uses Score Centering with top-32 rollout scores, no IS, and no extra sequence or token rejection. It uses the same learning-rate schedule as REINFORCE Zero (lag 2). Table 1 compares all recipes at 6,000 updates with the same 8K response budget.

Sequence rejection reference. All sequence gates use $\gamma = 10^{-2}$. For REINFORCE Zero and Sync TII, the gate is applied only as a diagnostic and rejects no sequences.

Mathematical reasoning without tools: evaluation. We evaluate MATH500 (500 problems), AMC23 (40), OlympiadBench (674), AIME25 (30), and Minerva Math (272) with temperature 0.6 and top-p 0.95. Figure 1a and Table 1 use an 8,192-token response budget with up to 16 responses per problem. Macro pass@1 averages the five benchmark scores with equal weight.

High-lag reasoning with Python tools. Non-thinking Qwen3-4B combines TII with token-level IS at lag 64, with 128 samples per update, a 16,384-token context, and at most 14,336 generated tokens.

Training spans 6,000 updates; Figure 6 shows reward and evaluation through 5,000 updates. AIME24, AIME25, and AIME26 each contain 30 problems.

Mathematical reasoning with Python tools. This task uses non-thinking Qwen3-30B-A3B and a 7,500-problem DAPO-Math subset, allowing at most ten Python tool turns. Training uses 16 GB200 GPUs and batches of 64 prompts with four responses each. The asynchronous recipe combines TII with group-mean advantages and token-level IS; R3 aligns expert selection, and the sequence gate is disabled. Evaluation uses temperature 0.7, top-p 0.7, and 8–32 responses per problem. The evaluation context windows include both prompt and response tokens (Table 3).

Visual-agent training on OSWorld. Each update trains on 128 episodes sampled from a 2,690-task computer-use training pool, with trajectory rewards supplied by the environments. The agent environments run on a CPU cluster for parallel interaction. The learner uses two nodes; two separate rollout nodes host eight vLLM engines with tensor-parallel size one. Rollout generation and policy updates proceed asynchronously, with routing replay enabled for the MoE layers. Training uses AdamW and a reward baseline. The 49,152-token context budget covers text, image tokens, and generation. Tasks span browsers, office applications, code editors, and other GUI applications.

We evaluate the policy after 428 updates at temperature 0.6 and top-p 0.95, with four samples per task. Task-weighted pass@1 averages the sample outcomes within each task, then averages across tasks.

A.3. Diagnostic definitions

Sampled rollout-to-learner KL estimate. For each sampled token a_t and history h_t, the rollout stores its generation log-probability $\log \mu_v(a_t \mid h_t)$. The learner scores the same token and history under its policy π_{θ_k} before the optimizer step. The sampled rollout-to-learner KL estimate is

$$\widehat{D}_{\text{rollout}\|\text{learner}} = \overline{\log \mu_v(a_t \mid h_t) - \log \pi_{\theta_k}(a_t \mid h_t)}, \tag{9}$$

where the overbar averages valid action tokens and $v \le k$ indexes the rollout weights. At a fixed history, this signed sample mean estimates $D_{\mathrm{KL}}(\mu_v \| \pi_{\theta_k})$.

Bernoulli KL diagnostic. The rollout-to-learner Bernoulli KL compares the rollout distribution with the current learner policy at each sampled action. For sampled-action probabilities u and z, respectively, clamped to $[10^{-6}, 1 - 10^{-6}]$, it averages

$$d_{\mathrm{Bern}}(u, z) = u \log \frac{u}{z} + (1 - u) \log \frac{1 - u}{1 - z} \tag{10}$$

first over valid tokens in each response, then over responses.

Importance-weight deviation. We measure importance-weight deviation with

$$D_3 = \overline{\tilde{w} - 1 - \log \tilde{w}}, \tag{11}$$

where $\tilde{w} > 0$ is the per-token importance weight used to compute the diagnostic, and the overbar averages valid action tokens in each update. Each contribution is nonnegative and vanishes at $\tilde{w} = 1$. For the exact ratio $w = \pi_{\theta_k}/\mu_v$ at a fixed history with common positive support, $\mathbb{E}_{\mu_v}[w] = 1$, so $\mathbb{E}_{\mu_v}[w - 1 - \log w] = D_{\mathrm{KL}}(\mu_v \| \pi_{\theta_k})$, the conditional divergence estimated by Equation (9).

Gradient norms. We report the ℓ_2 norm of the loss gradient before clipping.

B. Proofs and supporting calculations

B.1. A Gaussian analysis of update variance and stability

We derive the mean and variance of an update, then track squared error over training.

Setup. The target policy and reward are

$$p_\theta = \mathcal{N}(\theta, \sigma^2), \qquad R(a) = -\tfrac{1}{2}(a - a_\star)^2, \qquad J(\theta) = -\tfrac{1}{2}\big[(\theta - a_\star)^2 + \sigma^2\big],$$

with fixed $\sigma > 0$. Write $x_k = \theta_k - a_\star$ and $V_k = \mathbb{E}[x_k^2]$. The expected reward loss relative to the optimum is $J(a_\star) - \mathbb{E}[J(\theta_k)] = V_k/2$.

Each batch independently draws $t_k = \pm\kappa$ with equal probability, where $\kappa \geq 0$ is fixed. Given this sign, all M actions are sampled independently from the same policy $\mu_k = \mathcal{N}(\theta_k - \sigma t_k, \sigma^2)$. The importance ratio uses this realized sampling density. With the baseline $J(\theta_k)$ held fixed during differentiation, the update is

$$g_k(a) = \frac{p_{\theta_k}(a)}{\mu_k(a)}[R(a) - J(\theta_k)]\frac{a - \theta_k}{\sigma^2}, \qquad \theta_{k+1} = \theta_k + \eta\bar{g}_k, \quad \bar{g}_k = \frac{1}{M}\sum_{i=1}^{M} g_k(a_i). \tag{12}$$

Here $\eta > 0$ and $M \geq 1$. Expectations $\mathbb{E}_k$ and variances Var_k condition on the past: x_k is fixed, while the batch sign and actions are random.

Step 1: the mean update. For this batch, abbreviate $x = x_k$ and $z = (a - \theta_k)/\sigma$. Expanding the centered reward and multiplying by the policy score gives

$$R(a) - J(\theta_k) = -x\sigma z - \frac{\sigma^2}{2}(z^2 - 1), \qquad f_x(z) := [R(a) - J(\theta_k)]\frac{z}{\sigma} = -xz^2 - \frac{\sigma}{2}(z^3 - z).$$

The update is $g_k = w f_x$, where $w = p_{\theta_k}/\mu_k$. For either sign of t_k, the importance weight changes the expectation from the sampling policy to the target policy:

$$\mathbb{E}_k[g_k \mid t_k] = \int p_{\theta_k}(a) f_x\big((a - \theta_k)/\sigma\big)\, da = \mathbb{E}_{Z\sim\mathcal{N}(0,1)}[f_x(Z)] = -x.$$

The last equality uses $\mathbb{E}[Z^2] = 1$ and the vanishing odd moments of a standard normal. Averaging samples and signs therefore leaves $\mathbb{E}_k[\bar{g}_k] = -x_k$.

Step 2: the single-sample second moment. In the second moment, one factor of the importance ratio remains after canceling the sampling density:

$$\mathbb{E}_\mu[(wf)^2] = \int \frac{p(a)^2}{\mu(a)} f(a)^2\, da.$$

This is where mismatch affects the update even though its mean is correct. Let ϕ denote the standard normal density. Conditional on $t_k = t$, the standardized action z has density $\phi(z + t)$. Completing the square transforms the weighted integral into a normal moment:

$$\frac{\phi(z)^2}{\phi(z + t)} = e^{t^2}\phi(z - t), \qquad \mathbb{E}_k[g_k^2 \mid t_k = t] = e^{t^2}\mathbb{E}_{Z\sim\mathcal{N}(t,1)}[f_x(Z)^2].$$

The same density identity, integrated without f_x^2, gives $\mathbb{E}_{\mu_k}[w^2] = e^{\kappa^2} = 1 + \chi^2(p_{\theta_k}\|\mu_k)$.

To evaluate the update second moment, expand

$$f_x(z)^2 = x^2 z^4 + x\sigma(z^5 - z^3) + \frac{\sigma^2}{4}(z^6 - 2z^4 + z^2).$$

For $Z \sim \mathcal{N}(t, 1)$, the required even moments are

$$\mathbb{E}[Z^2] = t^2 + 1, \qquad \mathbb{E}[Z^4] = t^4 + 6t^2 + 3, \qquad \mathbb{E}[Z^6] = t^6 + 15t^4 + 45t^2 + 15.$$

The middle term has expectation $x\sigma(t^5 + 9t^3 + 12t)$, which is odd in t and cancels when averaging $t = \pm\kappa$. Substituting the even moments yields $\mathbb{E}_k[g_k^2] = A_\kappa x_k^2 + \sigma^2 C_\kappa$, where

$$\begin{aligned} A_\kappa &= e^{\kappa^2}(\kappa^4 + 6\kappa^2 + 3), \\ C_\kappa &= \tfrac{1}{4}e^{\kappa^2}(\kappa^6 + 13\kappa^4 + 34\kappa^2 + 10). \end{aligned} \tag{13}$$

Step 3: the batch variance. All samples share the sign t_k, so we first condition on that sign. The samples are then independent, and averaging divides their variance by M. Their conditional mean is $-x_k$ for either sign, so the sign itself adds no variance to the batch mean. The law of total variance gives

$$\mathrm{Var}_k(\bar{g}_k) = \frac{1}{M}\mathbb{E}_k[\mathrm{Var}_k(g_k \mid t_k)] + \underbrace{\mathrm{Var}_k(-x_k)}_{0} = \frac{\mathbb{E}_k[g_k^2] - x_k^2}{M}.$$

Consequently,

$$\mathrm{Var}_k(\bar{g}_k) = v_\kappa x_k^2 + b_\kappa, \qquad v_\kappa = \frac{A_\kappa - 1}{M}, \quad b_\kappa = \frac{\sigma^2 C_\kappa}{M}. \tag{14}$$

The positive polynomial coefficients in Equation (13) show that both v_κ and b_κ increase with $\kappa \geq 0$; both also scale as $1/M$. At matched sampling, $v_0 = 2/M$ and $b_0 = 5\sigma^2/(2M)$.

Step 4: accumulated error and stability. The mean and variance now determine how squared error evolves.

Theorem 1 (Accumulated error and stability)**.** If $V_0 < \infty$, then $\mathbb{E}[x_k] = (1-\eta)^k\mathbb{E}[x_0]$ and

$$V_{k+1} = \rho V_k + c, \qquad \rho = (1-\eta)^2 + \eta^2 v_\kappa, \quad c = \eta^2 b_\kappa > 0. \tag{15}$$

Consequently, V_k tends to $c/(1-\rho)$ if $\rho < 1$, grows linearly if $\rho = 1$, and grows exponentially if $\rho > 1$. Its stability condition is $0 < \eta < 2/(1 + v_\kappa) = 2M/(M + A_\kappa - 1)$.

Proof. Separate the update into its conditional mean and zero-mean noise:

$$\bar{g}_k = -x_k + \varepsilon_k, \qquad \mathbb{E}_k[\varepsilon_k] = 0, \qquad \mathbb{E}_k[\varepsilon_k^2] = v_\kappa x_k^2 + b_\kappa.$$

Then $x_{k+1} = (1-\eta)x_k + \eta\varepsilon_k$. Taking expectations and iterating gives $\mathbb{E}[x_k] = (1-\eta)^k\mathbb{E}[x_0]$. For squared error, the cross term vanishes because x_k is fixed given the past and $\mathbb{E}_k[\varepsilon_k] = 0$. Thus

$$\mathbb{E}_k[x_{k+1}^2] = (1-\eta)^2 x_k^2 + \eta^2\mathbb{E}_k[\varepsilon_k^2] = \rho x_k^2 + c.$$

Taking expectations gives $V_{k+1} = \rho V_k + c$. Iterating this recurrence and summing the geometric series yields

$$V_k = \rho^k V_0 + c\sum_{j=0}^{k-1}\rho^j = \begin{cases} \rho^k V_0 + c\dfrac{1-\rho^k}{1-\rho}, & \rho \neq 1, \\ V_0 + kq, & \rho = 1. \end{cases}$$

If $\rho < 1$, the initial-error term vanishes and the sum tends to $c/(1-\rho)$. If $\rho = 1$, each step adds c. If $\rho > 1$, rewriting the solution as $\rho^k[V_0 + c/(\rho-1)] - c/(\rho-1)$ shows exponential growth, since the coefficient of ρ^k is positive. Finally,

$$\rho - 1 = \eta\big[\eta(1 + v_\kappa) - 2\big].$$

For $\eta > 0$, the condition $\rho < 1$ is therefore equivalent to $\eta < 2/(1 + v_\kappa) = 2M/(M + A_\kappa - 1)$. □

B.2. Numerical values in the mismatch example

Figure 3 uses $a_\star = 0$, $x_0 = 1$, $\sigma = 0.2$, $M = 1$, and $\eta = 0.6$ for both curves. For matched sampling, $\kappa = 0$, $A_0 = 3$, and $C_0 = 5/2$, giving

$$\rho_0 = 0.4^2 + 0.6^2(3-1) = 0.88, \qquad q_0 = 0.6^2 \times 0.2^2 \times \tfrac{5}{2} = 0.036.$$

The limiting squared error is $q_0/(1-\rho_0) = 0.30$. For $\kappa = 0.2$, Equation (13) gives $A_\kappa \approx 3.373892$ and $C_\kappa \approx 2.961331$, so

$$\rho \approx 1.014601, \qquad c \approx 0.042643, \qquad V_{200} = \rho^{200} + c\frac{\rho^{200}-1}{\rho-1} \approx 68.27.$$

The stable step-size threshold drops from $2/3$ to approximately 0.5928, so the fixed $\eta = 0.6$ crosses the stability boundary. Meanwhile, both curves have the same mean error $\mathbb{E}[x_k] = 0.4^k$: their different rewards arise from the accumulated update variance.